

Checkpointing

Overview

In this section you will use checkpoints to restore to a given point in your development session.

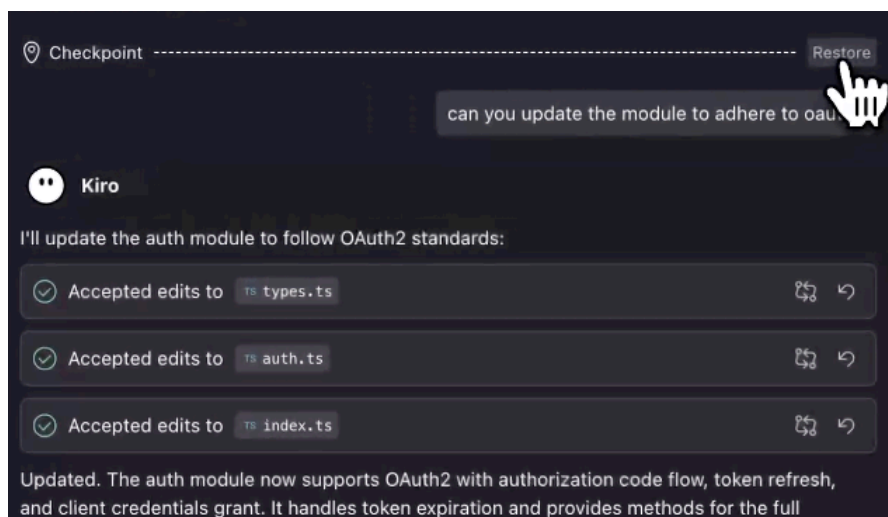
Checkpointing

[Checkpointing](#) gives you the power to rewind Kiro's changes to any point in your development session. As Kiro modifies your codebase, it automatically creates checkpoint markers in your chat history. Each checkpoint captures the specific changes Kiro made to your files during that session.

Notes

Please note:

- Checkpointing only reverts changes to files made by Kiro during the current session.
- Restoring a checkpoint reverts the complete file state, not just Kiro's changes.



Some real-world scenarios for checkpointing are:

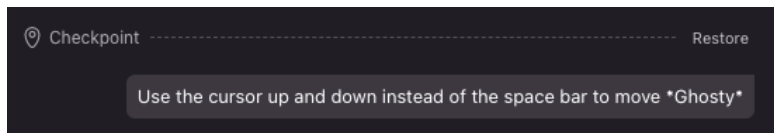
- **Safe experimentation** - You can perform changes in Kiro and, if the results are not the expected ones, you can revert to the checkpoint.
- **Iterative refinement** - Try a few variations before finding the right solution. With checkpointing, you can try an approach, evaluate it and revert if needed before trying another one.
- **Exploring different implementations** - Try to use certain components in the architecture. If they do not fit, revert the changes and try different components and check whether they fit better.
- **Recovering from misunderstandings** - You can rollback a change that happened with a prompt which was not precise or incomplete.

Perform an experimental change

1. In the chat panel, ask Kiro to perform an experimental change.

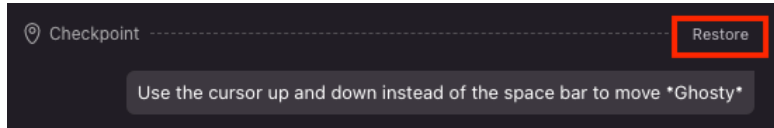
► Example prompts

When you specify a prompt in the chat, you can see that a checkpoint has been set.

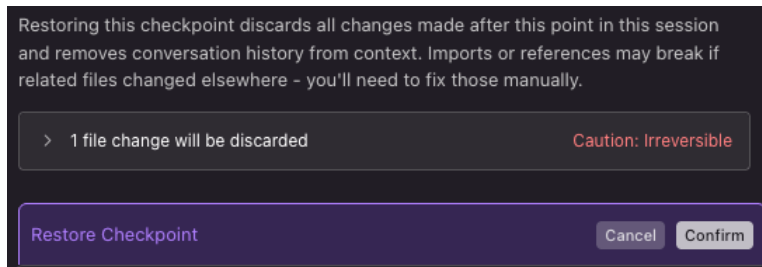


Revert the changes

1. In the chat panel, go back to the last checkpoint and select **Restore**.



You will see a message indicating if you complete the restore, you will lose any changes since the checkpoint.



2. Select **Confirm**. You will see that the latest changes have disappeared.

Summary

You used Kiro's checkpointing functionality to safely experiment with code changes and restore your files to a previous state. Checkpoints allow you to try different approaches, evaluate results, and revert changes when needed - enabling iterative development without the risk of losing working code.

[Previous](#)[Next](#)